# TIVIDY: Workflow Management System

**COS 214 Final Project, 2026** · Group 7MinInHeaven

Lushomo Mankomba: u25089464

Tsohle Masia: u25021967

Jordan Reddy: u25029666

Thandolwethu Jantjies: u25114469

Robert Sibanda: u25110782

Ashley Mthemba: u25072235

Gabriel van Staden: u25172922

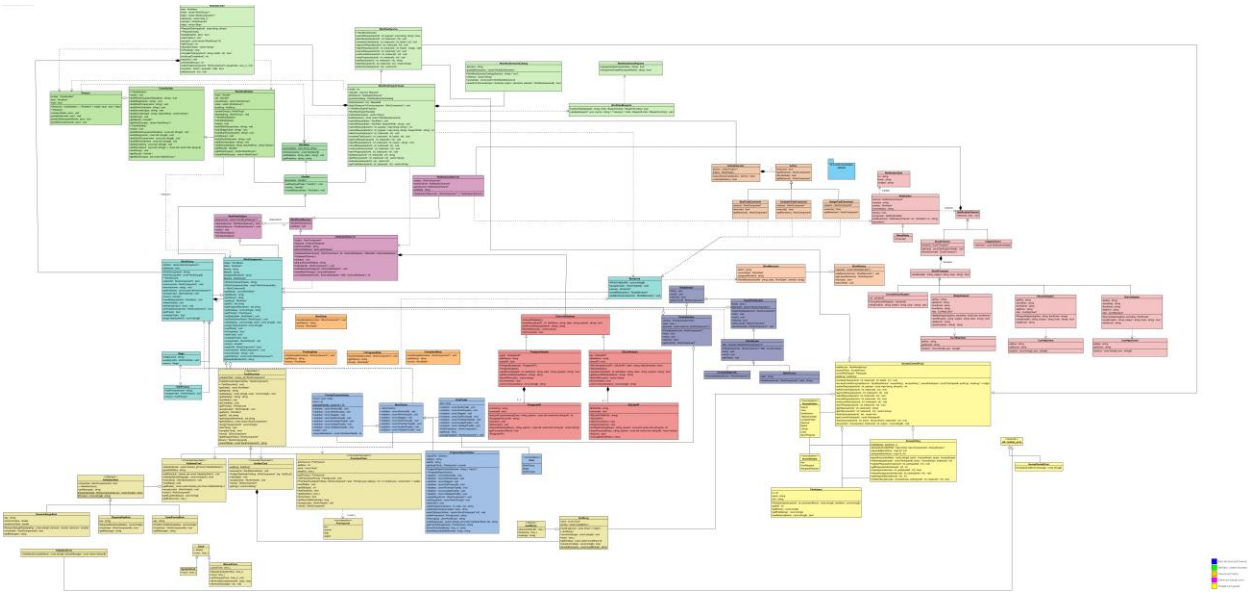
## Quick links

- **Repository: https://github.com/7MinInHeaven/TIVIDY_214**
- `README.md :` https://github.com/7MinInHeaven/TIVIDY_214/blob/dev/README.md
- ·DOCS : https://github.com/7MinInHeaven/TIVIDY_214/tree/dev/docs/FINAL%20Documentation
- `Report:` https://github.com/7MinInHeaven/TIVIDY_214/blob/dev/tests/AllPriorityPrioritisedTask.html
- Doxygen: https://7mininheaven.github.io/TIVIDY_214/annotated.html

## Contents

1. Team and contributions
2. Research, assumptions and boundaries
3. Requirements and traceability
4. Architecture overview
5. Subsystems
6. The two industries
7. Runtime behaviour
8. Build, run and test
9. Design decision log
10. Appendix A: GoF pattern mapping
11. Appendix B: References

**Diagram colour legend:** *state the colours for engine classes, industry classes and external systems.*



*https://github.com/7MinInHeaven/TIVIDY_214/blob/dev/docs/FINAL%20Documentation/UML%20joined.svg*

## 1. Team and contributions

| Member | Contribution |
|---|---|
| Lushomo | Designed and implemented the website integration, client and admin side. Designed the decorator and proxy strategies. Created C++ backend server implementation. Decorator adds dynamic additional responsibilities and Proxy provides authorization and restriction. |
| Tsohle | Implemented the notification subsystem. This system uses a NotificationObserver, a Notification Bridge implementation and Strategy combined with external adapters connected to emailing APIs (Brevo, Resend and Mailjet). |
| Jordan | Designed and implemented the chain of responsibility sub-system, allowing for Work Items to be processed sequentially by Composite items. Created Pending, InProgress, and Completed States for every task, so that after they are created commands can be used to progress the status of the task. Implemented Builder JSON parsing and dynamic creation and linking of classes given a JSON blueprint. |
| Thando | Designed and implemented the memento and command patterns to work together: a snapshot is taken before each action runs, which allows the action to be undone. Wrote tests for both patterns. |
| Robert | Designed and implemented the database subsystem: DatabaseObserver, ExternalDatabase strategy interface, PostgresAdapter + PostgresAPI(Cloud Neon PostgreSQL using libpq) and SQLiteAdapter + SQLiteAPI(local offline storage using sqlite3) with two-way database synchronization. Also was responsible for initial |

| Member | Contribution |
|---|---|
| | Docker setup(Dockerfile, docker-compose.yml) and Doxygen documentation setup. Database sequence diagram and database activity diagram. |
| Ashley | Implemented the CI/CD pipeline by configuring CMake, Docker, GoogleTest, GitHub Actions, and branch protection for main and dev, ensuring smooth and reliable continuous integration. I also implemented the Builder's dynamic runtime behaviour, allowing workflows to be created from JSON scenarios, and applied the Prototype pattern to clone the constructed workflow safely. |
| Gabriel | Designed and implemented the composite work model, where WorkGroup, Stage and Subprocesses are the composites and WorkUnit the leaf, with nesting validation, duplicate-name rejection and deep copy cloning. Implemented the iterator subsystem for depth-first traversal, state and incomplete task filtered searches and priority-ranked traversal. Implemented the Visitor subsystem which involves the nesting validator, the priority checker a progressReportvisitor that generates a HTML file progress report |

**Branching and review process:** The team followed a strict Git feature-branching workflow off the main integration branch(dev):

- Feature isolation: All development took place on dedicated branches (naming convention: feat/<feature-name>). Direct commits to main without reviews from at least 2 other members were not allowed.
- Automated verification: Every Pull Request triggered automated GitHub Actions CI builds compiling under C++11 compiler flags and executing the full 78+ GoogleTest suite and verifying zero memory leaks with Valgrind.

## 2. Research, assumptions and boundaries

**Findings:**

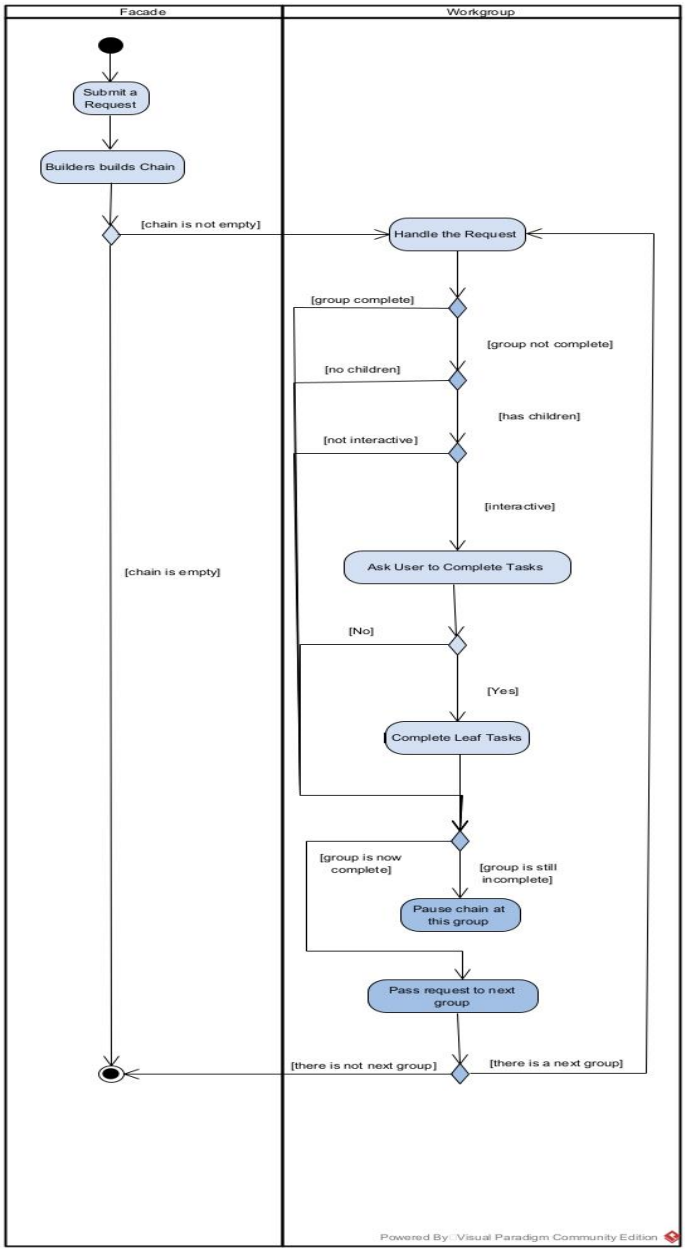
Our design was guided by 5 core research findings from workflow engineering literature and industry implementations:

1. Separation of Definition and Runtime instance (Workflow Management Coalition[3], Teiva Systems[2]): The Workflow Management Coalition reference model establishes that a process definition(build-time-schema) must be strictly decoupled from the running process instances(runtime execution)[3]. Teiva Systems demonstrate that storing process definitions in external configuration schemas rather than compiled C++ code allows workflows to be modified without rewriting the underlying engine [2].
    - Design impact: TIVIDY defines workflows using external JSON blueprints (service_blueprint.json, life_blueprint.json). The Builder Pattern (Director, workflowBuilder) parses the schema into master template graphs and the Prototype pattern (cloneWorkGroups()) deep copies the templates into completely isolated runtime instances with independant state and memory.

2. Hierarchical Processes (Workflow Management Coalition[3]): Industrial workflows are rarely ever flat in that they usually consist of nested groups, sequential statges, concurrent sub-processes and discrete work units [3].
    - Design impact: This hierachial nature is modlled using the Composite pattern(WorkComponent, WorkGroup, Stage, SubProcess, WorkUnit). The engine invokes operations (execute(), isComplete(), notify()) uniformly across single checklist tasks and entire multi-stage branches.
3. Tasks follow fixed status rules and have a lifecycle (Slack [1], Camuda [4]): Tasks must transition through explicit lifecycle states to ensure that business rules are enforced(for example an unassigned task cant be started and completed tasks cannot be modified).
    - Design impact: implemented with the State Pattern (WorkState, PendingState, InProgressState, CompeltedState) which encapsulates these rules. State transitions are validated polymorphically on the WorkComponent, each state decides which state comes next.
4. Keep a record of everything that happens(ScienceDirect [5], Camuda [4]): Modern workflow systems maintain an immutable audit trail of every status change, so there is a history for auditing and for recovering after failure.
    - Design impact: implemented using the Observer Pattern (DatabaseObserver) which saves a record everytime a task changes state or payload data combined with Adapter and stategy (ExternalDatabase, PostgresAdapater, SQLiteAdapter) that let that saved record go to a cloud database or local file. Tasks dont need to know which one is in use.
5. Actions must be reversible(ScienceDirect [5]): Humans make mistakes so a workflow must be able to undo a step or reopen a task without violating object encapsulation[5].
    - Design Impact: Implemented with the Command pattern (ActionExecutor, StartTaskCommand, CompleteTaskCommand, AssignTaskCommand) which treats each action as an object that can be undone, combined with the Memento pattern - (WorkMemento, WorkHistory), which saves a tasks state, allowing state restoration on rollback/undo.

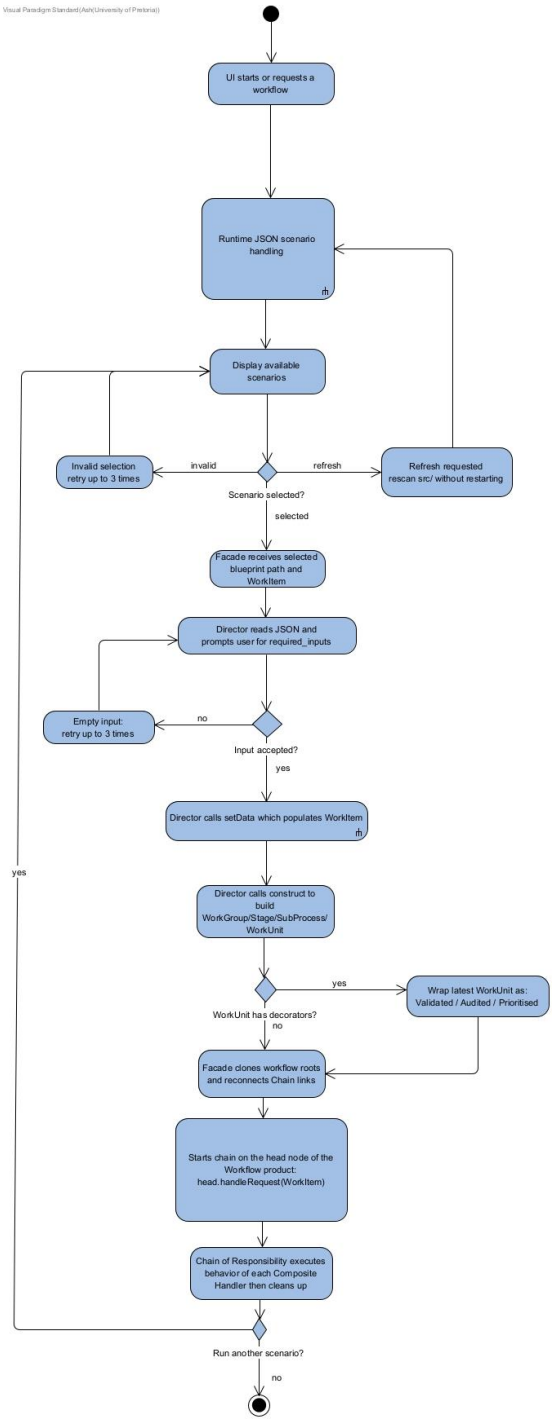
## 4. Generic vs industry

*Activity 1: generic execution of one instance*

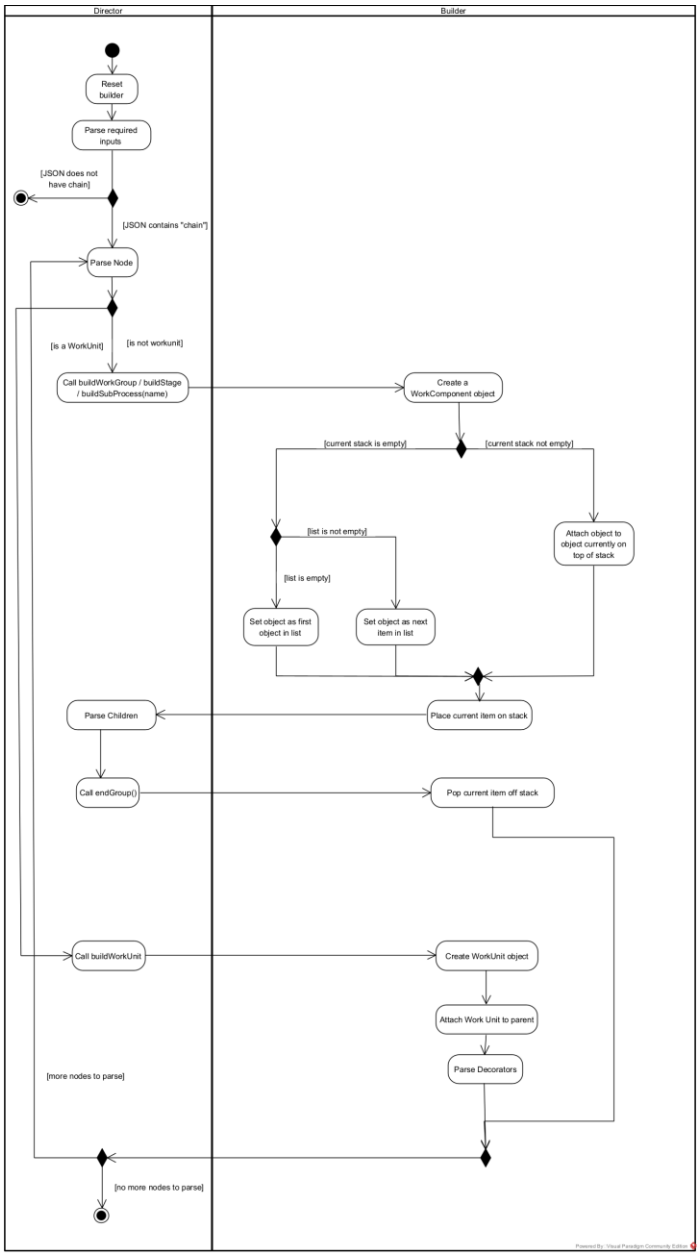



*Activity 2: another generic engine process*

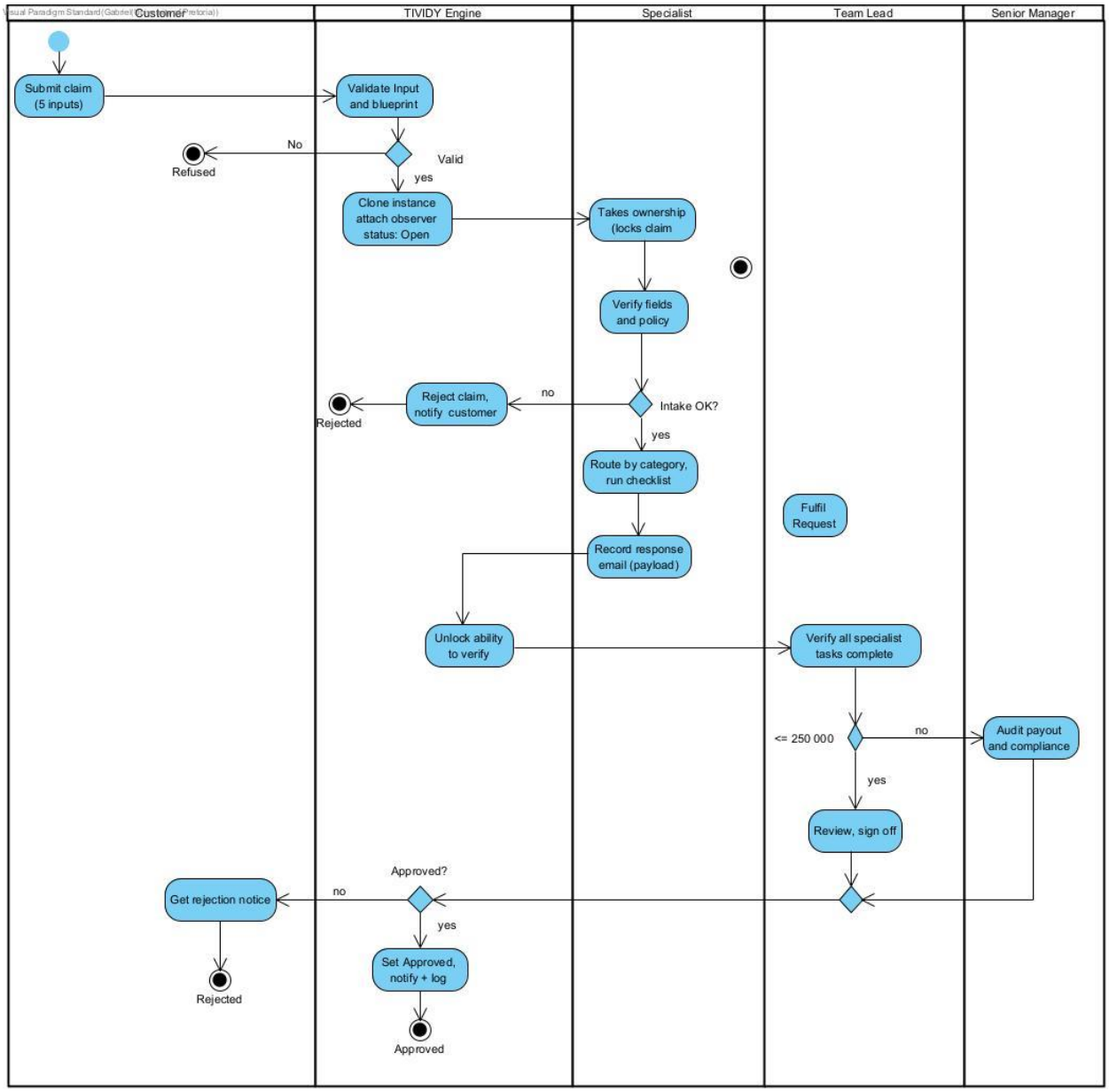
*First part of generic engine process (Builder Acitivity Diagram)*



*Second part of generic engine process*
*(Expansion of Director Builder build Action)*




*Activity Diagram 3: Industry Workflow*




## 5. Subsystems

*Same four* lines *for each: responsibility, how it works (small diagram), patterns and why, tests. Add or remove subsystems to match the code.*

### 5.1 Definition and instantiation

*Builder, Prototype, Facade*

My subsystem combines the Builder, Prototype, and Facade patterns to create and execute workflows dynamically at runtime.

The Builder constructs a complete workflow hierarchy from a JSON blueprint. The Facade provides the main entry point into the system, while the Prototype functionality clones the constructed workflow so that each request receives its own independent workflow instance.

## Responsibility

My subsystem is responsible for allowing the workflow engine to create, validate, select, and execute different workflow scenarios at runtime.

It allows new JSON scenarios to be added to the src/ directory and selected without recompiling or restarting the application. It also ensures that each request receives an independent copy of the workflow, preventing different requests from sharing or corrupting the same workflow state.

## How it works.

- The WorkflowScenarioCatalog scans the src/ directory for JSON workflow files and validates them through WorkflowBlueprint. Invalid files are rejected and reported without crashing the application.

- When a scenario is selected, the WorkflowEngineFacade coordinates the process. It passes the validated JSON blueprint to the Director, which instructs the WorkflowBuilder to create the required WorkGroups, Stages, SubProcesses and WorkUnits.

- The Builder then connects the objects through the existing Chain of Responsibility. It can also apply supported decorators defined in the JSON file.

- Once the workflow structure has been built, the Prototype functionality deep-clones the workflow. The cloned workflow is then executed independently, while the original built structure can be safely discarded or reused for another request.

## Why these patterns.

- **Builder:**
  I used the Builder pattern because workflows can have different structures and levels of complexity. Instead of hard-coding every possible workflow, the Director reads the JSON blueprint and uses the Builder to construct the correct hierarchy step by step.

- **Prototype:**
  I used the Prototype pattern so that each request receives an independent copy of the constructed workflow. This prevents shared state between requests and allows the same workflow definition to be reused safely.

- **Facade:**
  I used the Facade pattern to hide the complexity of the entire workflow process. The user interacts with one simplified interface while the Facade coordinates scenario discovery,

validation, input collection, Builder construction, cloning, Chain execution, observers and notifications.

**Tests:**
tests/test_builder.cpp verifies:

- Construction of nested composite workflow hierarchies.
- Resetting the Builder.
- Deep cloning of independent workflow chains.
- Applying decorators to WorkUnits.
- Rejecting invalid decorator requests.
- Constructing workflows from JSON.
- Populating required inputs into a WorkItem.
- Retrying empty user input.
- Rejecting repeated invalid input.
- Handling null builders and multiple WorkGroups.
- tests/test_facade.cpp verifies:
-
- Submitting the default workflow without errors.
- Executing a selected JSON blueprint.
- Listing runtime-discovered workflow scenarios.
- These tests confirm that my subsystem can dynamically construct, clone, select and execute workflows while preserving the existing workflow patterns.

## 5.2 Work model and lifecycle

*Composite, State, Iterator, Visitor*

**Responsibility.**

Defines what a work is, how it is structured and how it goes from start to finish. Every Workflow is structured as a tree of work components. Every component is Pending, In Progress Or Complete, and only legal transitions are allowed. Clients can traverse the node through all Components, or filter by state or incomplete only, or by priority. Nesting validation and HTML progress Report are added without changing the WorkComponent's

**How it works.**

- WorkComponet is the common interface. WorkGroup, Stage and SubProcess are the composites, and WorkUnit the leaf. Composites own their children and handles their deletion in its destructors. Add() rejects children that is not lower in hierarchy then parent and rejects children who's name matches a sibling. Each component gets a randomly generated 8-digit hex ID. Cloning the tree makes a deep copy of the whole tree
- State is attached to each task to see if it is Pending, In Progress or Completed. As tasks are traversed through the Chain of Responsibility, they receive commands which can transition the tasks toward a completed state. We can then see the competition or failure of a Workflow based on the total of all of the tasks states.
- Iterators are external and make use of the WorkIterator Abstract Interface. DepthFirstIterator traverses the tree in post-order. FilterIterator adds on top of it and skips node that do not follow a specific condition. StateIterator and Incomplete Iterator provide the necessary filters. PriorityIterator collects prioritised tasks and sorts them by rank, with overdue tasks pushed to the top
- Each component implements accept(WorkVisitor) which calls the matching visit() overloading. KindFinder classifies component so that WorkComponentValidator can check nesting without type checks. PriorityCheckerVisitor detects and ranks prioritised tasks for PriorityIterator. ProgressReportVisitor writes HTML report about the tree with collapsible groups, progress circles and details pertaining to the decoration of Components

**Why these patterns.**

- **Composite:** Workflows need to be nested and engine needed to treat single task and whole stage the same way
- **State:** State needed to track the progress of tasks. Commands can be used to represent tasks being done.
- **Iterator:** Engine need to walk tree in different ways. Iterators keep traversal logic out of clients and let traversal be swapped without exposing how tree stored
- **Visitor:** Need to add functionality to Components without bloating the Componet class. Replace the use of dynamic_cast and make the building of a report less cumbersome

**Composite:**

- Add valid child = added, parent set
- Add Incorrect nesting = rejected
- Add child with duplicate sibling name = rejected
- Remove child = Child deleted, group state refreshed
- Clone and change clone = Original unchanged

**State:**

- Transition state = Only valid state transition allowed

**Iterator:**

- Empty or leaf-only tree = isDone behaves correctly
- State and Incomplete filters = only return matching nodes

- Priority ordering with overdue task = Correct ordering of priorities with overdue placed higher

**Visitor:**

- Nesting validation with decorated task = Traverse down to wrapped task visit
- Generate report = File contains correct group names and details

## 5.3 Routing and approvals

*Chain of Responsibility*

**Responsibility.**

Links steps in a workflow together. WorkGroups represent steps and can have nested sub-processes, which are linked in a vector so that when a WorkItem needs to be processed it traverses all the WorkGroups in the vector.

**How it works.**

Handler is the abstract class that WorkComponent (the Concrete Handler) inherits from, this defines how links in the chain are linked together and how a WorkItem can pass through the entire chain. We link WorkGroups through the setNext function. Items pass through the chain by constantly calling the base Handler::handleRequest allowing for the item to trickle down all the Workgroups and their sub-processes and be edited by any process it encounters.

**Why this pattern.**

Workflow steps must run in fixed order and a step must be able to block next one until it is finished. Handler can either handle request or pass it on, allowing for steps to be skipped which is necessary for a dynamic workflow system. Client only sends requests to first group, allowing for steps to be added or reordered without client knowledge

**Tests.**

Group with no successor = handles request alone

Set next = Adds a next step that handler is able to traverse

## 5.4 Actions, history and recovery

*Command, Memento*

**Responsibility.**

This part of the engine makes specialists' actions traceable and reversible. Each action a specialist takes on a task (assigning it to a worker, starting it, completing it) is recorded, and the most recent action can be undone, returning the task to exactly the state it was in. The Command pattern is responsible for running and recording actions. The Memento pattern is responsible for saving and restoring task state. Neither pattern knows how a task works internally, and the tasks contain no undo logic.

**How it works.**

- Each specialist action is a command object (AssignTaskCommand, StartTaskCommand, CompleteTaskCommand). The Facade checks that the claim is open and that the specialist owns it, then passes the action to the ActionExecutor, which runs it and keeps the history.
- Before a command runs, the executor asks the task for WorkMemento, a snapshot of its id, state and assigned worker, and stores it in WorkHistory. If thhe command succeeds, both are kept. If it fails, both are discarded and the task is unchanged.
- Completing a pending task runs assign, start and complete as one step. If any is refused, the earlier ones are rolled back, so a task is never left half-finished.
- To undo, the executor restores the task from the latest snapshot and removes the snapshot and the command from their histories. A command can also declare itself not undoable, and the executor will then refuse to undo past it

**Why these patterns.**

- **Command:**
  We used the command pattern so every action a specialists does regarding the tasks can be saved

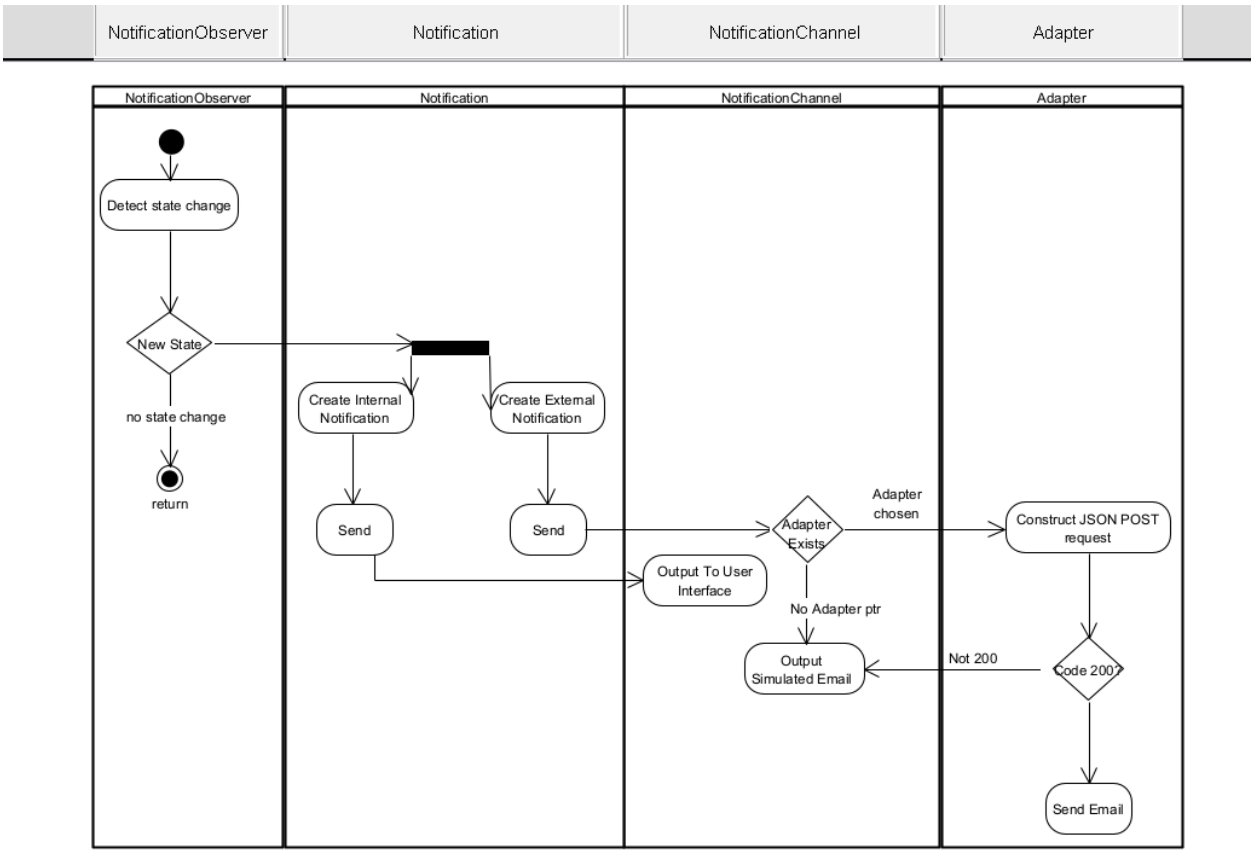
- **Memento:**
  We used the memento pattern to add roll back functionality so if a specialist either makes a mistake or wants to undo an action, it is possible

**Tests.**

- **Command:** each command is run directly on a task. AssignTaskCommand is checked to report success and to set the assigned worker. StartTaskCommand is run on a task that already has a worker, and CompleteTaskCommand is run on a task that has been started. Both are checked to report success.
- **Memento:** a task is given a worker, a snapshot is taken, and the worker is then changed. Restoring the snapshot is checked to bring the original worker back, which shows that a task can be rolled back to the state it had when the snapshot was taken.

## 5.5 Events and integration

*Observer, Adapter, Bridge, Strategy*




**Responsibility:**

- Create a way in which a user may be notified of the progression of the status of their request via in app channels and external email channels.

**How it works:**

- The Observer detects a change in state of the request item. If the state remains the same, the algorithm exits. If there is a change an internal and external notification is created.
- The internal notification simply outputs to terminal/ use the User Interface. The external firstly checks whether the user has selected their adapter strategy of choice. If not, the programme prints a simulated email to terminal. If there is one, a JSON string is created and is then used to create a POST request to the selected API.
- Should this fail, the programme falls back to a terminal/ internal output.

**Why these patterns:**

- The ***Adapter*** allows the client-side code to connect to an existing API without having to interfere with the existing APIs logic/ internal structure.
- The ***Strategy*** provides flexibility and allows the user to pick any API they see fit without having to change internal logic every time but rather assigns the pointer to a different Strategy.

- The ***Bridge*** allows the Notification to not need to know how to send it but rather allows the Implementors, that being the Notification channels (with differing implementation) to have the logic to do so. The ***Observer*** detects any changes in the state of the request and sends an appropriate notification.

**Test:**

- Checked to see if POST request returns 200 else revert to terminal output.

### 5.6 Access control

*Proxy*

**Responsibility.**

Enforces role-based access control and concurrency locks, ensuring participants can only perform actions they are authorized for, and preventing two agents from working on the same request simultaneously.

**How it works.**

The AccessControlProxy sits between the UI (Website/TUI) and the WorkflowEngineFacade. Before forwarding any request to the engine, it checks the AccessPolicy. If the action is denied (e.g., a customer trying to approve a claim), it throws an AccessDeniedError and logs it to the AuditLog. It also locks the request to the first agent who calls takeOwnership, blocking other agents from modifying it. The Proxy only forwards the request to the Facade if all checks pass.

**Why this pattern.**

We used the Proxy pattern to decouple security from the core engine. The workflow engine shouldn't need to know about user roles or locks. The Proxy acts as a protective barrier, keeping the engine focused strictly on executing the workflow, while also providing a single point to enforce security.

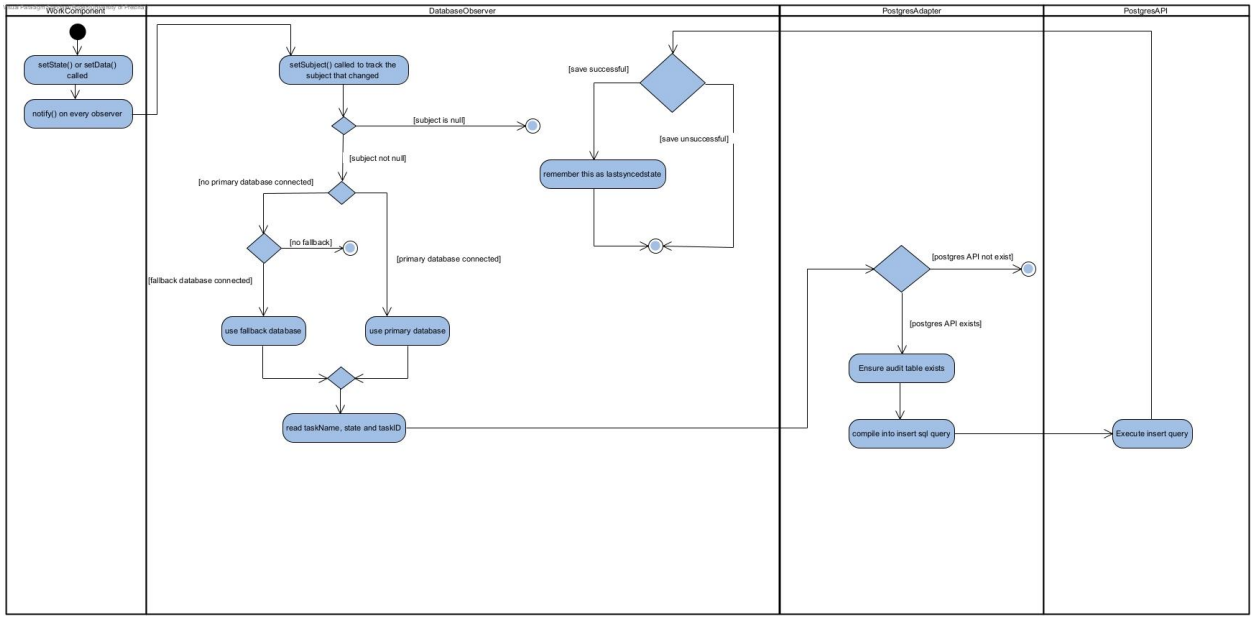
**Tests.**

tests/test_proxy.cpp verifies:

- Customers can only see and cancel their own requests.
- Specialists can take ownership and complete tasks, but only on claims they own.
- A second agent cannot take or modify a claim already owned by another.
- Role-based denials are logged correctly.
- The same AccessPolicy configuration works for two completely different industries (Insurance and Service Requests).

### 5.7 Auditing and persistence

*Adapter, Strategy and Observer*




**Responsibility:** Provide a way to track and save progress in a workflow in order to keep track of what was done and when.

**How it works:** A DatabaseObserver watches the entire tree of tasks and stages. Whenever any task changes state like from Pending to In Progress it is notified and pulls the task's state and user payload and taskID, and saves it. DatabaseObserver only talks to a single ExternalDatabase interface, which acts as both our Strategy and our Adapter Target. Our two adapters adapt two incompatible database APIs into that one interface: PostgresAdapter adapts PostgresAPI which wraps libpq cloud calls like executeTransaction for our live Neon PostgreSQL database while SQLiteAdapter adapts SQLiteAPI, which wraps local sqlite3 file calls like runStatement for our offline database. Because of the Strategy pattern, if the cloud connection drops, DatabaseObserver automatically falls back to local SQLite so the workflow never crashes, and syncDatabases() syncs missing rows both ways between local and cloud without duplicates.

**Why these patterns:**

Observer decouples our core workflow engine from the storage logic. WorkComponents simply notify attached observers when their state or payload changes and the engine doesn't need to know anything about databases, tables or SQL queries.

Adapter bridges the gap between the incompatible database libraries. PostgreSQL (libpq) and SQLite(sqlite2) have completely different low-level C function calls, connection handles and execution flows. The adapter pattern translates both into one uniform c++ interface so the rest of the application never deals with raw database drivers.

Strategy makes dynamic backed switching possible. By treating ExternalDatabase as an interchangeable persistence strategy, DatabaseObserver can evaluate connection health at runtime using the selectBestStrategy() function. If the remote cloud connection drops the system falls back to saving locally in SQLite without crashing or losing data. It also allows the 2 strategies to be swapped on the fly and synchronized both ways using the syncDatabases() function.

**Tests:**

- tests/test_databaseadapters.cpp: Verifies mock and live PostgreSQL transactions, SQLite local table creation and row insertion, automatic cloud-to-local fallback, multi-subject tree observation with a single observer, and two way synchronization (syncDatabases).
- tests/test_observer.cpp: Tests WorkflowSubject registration (attach, detach), notification broadcasting, and state synchronization across multiple attached observers.

## 6. The two industries

### 6.1 Industry 1: Water Leak Repair (municipal services)

A university campus facilities department (TIVIDY Campus Facilities Management) where residents report maintenance faults online on the Customer View and maintenance dispatchers, plumbers and residence wardens triage, repair and signoff tickets from their queue (in the supportview), governed by role based access control through the AccessControlProxy. Maintenance operations are handled in 4 structured phases:

- Residence Helpdesk and Triage: intake verification and water damage assessment risk
- Facilities Dispatch and stores: maintenance plumber assignment and central stores parts requisition.
- On-site Plumbing and Testing: physical valve isolation, cartridge replacement and pressure leak verification.
- Handover and signoff: resident confirmation, labor and material cost logging and ticket closure.

**Process being managed:**

The end-to end handling of a student residence broken tap and water leak repair request(src/service_blueprint.json):

1. Logging the ticket, verifying student residence standing and evaluating emergency flood risk(WorkGroup 1: Residence Helpdesk Intake & Leak Triage).
2. Assigning an on duty campus plumber, scheduling room accessa and requesting replacement plumbing parts from central stores(WorkGroup 2: Facilities Dispatch & Parts Requisition).
3. Executing on-site isolation, disassembling the tap, refitting the valve seat/cartridge and conductiong live water pressure leak tests (WorkGroup 3: On-Site Plumbing Repair & Pressure Testing).
4. Logging maintenance costs against the residence budget, securing student sign-off and sending the completion email notice (WorkGroup 4: Student Inspection Sign-Off & Ticket Closure)

**Participants and roles involved:**

| **Role** | **Responsibility in TIVIDY** |
|---|---|
| Student/Resident (Role::Requester) | Submits the repair ticket with 5 required input fields (student_number, email, residence_room, urgency_level, issue_description), views live |

| | |
|---|---|
| | repair status/history, or cancels an open request (cancelRequest) |
| Helpdesk Dispatcher (Role::Specialist) | Takes ownership of the open maintenance ticket (takeOwnership), verifies the student is registered to the reported building, and assesses emergency water damage risk (WorkGroup 1). |
| Campus Plumber (Role::Specialist) | Checks out tools and parts from central stores (WorkGroup 2), performs the physical on-site repair, and conducts live water pressure flow tests (WorkGroup 3). |
| Maintenance Supervisor/ Warden (Role::TeamLead/Manager) | First- and final-tier facilities approver (WorkGroup 4); audits parts used and labour hours, verifies resident satisfaction, and formally closes/approves the ticket (approveRequest). |

**Input data:**

When a student submits a repair request , Director/WorkflowEngineFacade populates the WorkItem payload map with 5 required keys defined in src/service_blueprint.json:

| Key | Example | How engine uses it |
|---|---|---|
| Student_number | "u23456789" | Validated in Stage 1A against university student housing records to ensure the student lives in campus accommodation; persisted in audit rows. |
| email | "student@up.ac.za" | Validated in Stage 1A. Read by WorkflowEngineFacade to dispatch confirmation and completion notices via EmailChannel. |
| Residence_room | "House Asterhof, Block B, Room 204" | Read in Stage 1A and Stage 2A to schedule the repair and coordinate room access with residence reception. |
| Urgency_level | "Flooding" | Read in Stage 1B: Water Damage Risk Assessment (SubProcess 1B-1) to determine whether the main building |

| | | |
|---|---|---|
| | | stopcock must be shut down and alert the House Parent. |
| Issue_description | "Hot water tap handle spins loosely and water is leaking into basin cabinet" | Checked in Stage 1A and read by plumbers in Stage 2B to identify required replacement parts (cartridge vs. full mixer tap assembly). |

**Decisions and routing points:**

1. Intake & Location Verification (WorkGroup 1): Verifies that student_*number and that all fields are non-empty. If any fields are empty* intake rejects the ticket.
2. Emergency Flooding Triage (SubProcess 1B-1): Evaluates urgency_level:
   - Dripping / SteadyStream: Standard dispatch queue.
   - Flooding: Triggers immediate emergency escalation to shut off the main block stopcock and notify the Residence House Parent.
3. Parts Requisition Branch (SubProcess 2B-1): Plumber evaluates the severity described: retrieves ceramic cartridges and O-rings for internal reseating, or checks out a full mixer tap assembly if the casting is cracked.
4. Live Pressure Leak Test (SubProcess 3C-1): Restores line pressure to test for zero dripping:

- Pass: Advances to handover and sign-off (WorkGroup 4).
- Fail (secondary leak detected): Plumber reopens isolation tasks to replace damaged seals.

**Approvals and dependencies:**

- Stage-to-Stage Dependency (RequestChain::unfinishedGroup): Plumber dispatch (WorkGroup 2) cannot begin until Helpdesk triage (WorkGroup 1) completes. On-site plumbing (WorkGroup 3) cannot execute until parts are checked out (WorkGroup 2), and final sign-off (WorkGroup 4) is locked until pressure testing succeeds.
- Ownership Dependency (takeOwnership): Only the assigned helpdesk agent or plumber who owns the ticket can execute checklist commands, preventing multiple staff from duplicating work.
- Handover Sign-Off Approval: Closing the ticket requires student or warden confirmation alongside supervisor sign-off on parts and labour budgets (approveRequest).

**Completion path:**

1. Ticket Submission (submitRequest): The student (Role::Requester) submits the 5 fields. WorkflowBlueprint::loadAndValidate("src/service_blueprint.json") validates the schema, Director and WorkflowBuilder assemble and deep-copy clone (cloneWorkGroups()) an isolated 4-group tree, stampPayload copies the inputs into all components, and attachObserverToTree attaches DatabaseObserver. The ticket starts as "Open".
2. Ticket Ownership (takeOwnership):
   A helpdesk dispatcher (Role::Specialist) takes ownership of the ticket (request.owner =actorId).
3. WorkGroup 1 - Intake & Leak Triage Completion (completeTask):
   The dispatcher logs the ticket, verifies student registration and room location (Stage 1A), and

evaluates water damage risk (Stage 1B). State transitions fire notify(), logging Pending → In Progress →Completed into PostgreSQL and SQLite. Once all tasks finish, WorkGroup 1 automatically marks itself Completed, unlocking WorkGroup 2.

4. WorkGroup 2 - Dispatch & Parts Requisition Completion (completeTask):
   A plumber is assigned, access is scheduled (Stage 2A), and replacement ceramic cartridges, washers, and basin spanners are checked out from central stores (Stage 2B). WorkGroup 2 completes, unlocking WorkGroup 3.
5. WorkGroup 3 - On-Site Repair & Testing Completion (completeTask):
   The plumber isolates the angle valve, disassembles the tap (Stage 3A), cleans the brass seat and installs the replacement cartridge (Stage 3B), then restores pressure and verifies zero dripping (Stage 3C). WorkGroup 3 completes, unlocking WorkGroup 4.
6. WorkGroup 4 - Handover, Cost Logging & Ticket Closure (completeTask + approveRequest):
   Labour hours and parts costs are recorded against the residence maintenance budget (Stage 4A), resident confirmation is obtained, and the supervisor calls approveRequest(actorId, instanceId), marking the ticket "Approved" / completed.
7. Notification & Audit Trail:
   ResultNotice is dispatched to student@up.ac.za via EmailChannel (or ConsoleEmailAdapter), and all state changes and task metadata remain stored in workflow_audit_log in the database.

**Exception paths and possible failures:**

- Empty or Invalid Submission (invalid_argument): Omitting required fields or submitting empty strings is rejected immediately during intake.
- Unauthorised Role Action: A student attempting to dispatch a plumber or mark a task complete is blocked by AccessControlProxy.
- Out-of-Order Execution: Attempting to start on-site repair before dispatch and parts requisition finish is rejected by RequestChain::completeTask().
- Ticket Rejection (rejectRequest): If the student is not a registered resident of the reported hall, the ticket is formally marked "Rejected" with a logged reason.
- Ticket Cancellation (cancelRequest): If the resident resolves the leak themselves before dispatch, the student cancels the ticket, locking further transitions.
- Mistaken Task Completion (undoLastAction): If a plumber accidentally marks the repair complete before pressure testing, undoLastAction() uses ActionExecutor and WorkMemento to roll the task back from Completed to Pending.
- Cloud Database or Email Outage: If Neon Cloud PostgreSQL is offline, DatabaseObserver::selectBestStrategy() routes writes to local SQLite (tividy_local.db); missing email credentials fall back gracefully to ConsoleEmailAdapter.

**Expected outcomes of workflow:**

- The maintenance request advances from "Open" to "Approved" (or "Rejected" / "Cancelled" on an exception path).
- The tap is repaired and pressure-tested, with all costs accounted for.
- Every task transition is logged with its task_id and student room payload in both Neon Cloud PostgreSQL and local tividy_local.db.
- The student receives automated confirmation and completion notices via email.

## 6.2 Industry 2: Life Insurance Claims

A life insurance provider (TIVIDY Life Insurance) where customers submit claims online via the Customer View and specialist claims agents process, assess, and approve payouts from their queue (Support View), governed by role-based access control (AccessControlProxy). Claims processing teams are divided into 3 specialized units:

- Death Benefits Team (natural passing of policy holders)
- Accidental Claims Team (severe accidents, fatal trauma, dismemberment)
- Critical illness and Disability Team (living benefit payouts and permanent disability)

Process being managed: The end-to-end handling of an incoming life insurance claim.

1. Validating the 5 required claim inputs and checking policy standing(IntakeHandler)
2. Routing to the appropriate specialist team based on claim category and completing the category's specialist investigation checklist(AssessmentHandler)
3. Enforcing two-tier financial sign-off based on whether the claimed amount is up to R250 000 or above it, then dispatching the final outcome notification.(ApprovalHandler)

**Participants and roles involved:**

| Role | Responsibility in TIVIDY |
|---|---|
| Customer/Claimant(Role::Requester) | Submits the claim with the 5 required input fields(policy*number, customer_email, category, amount, description), views status/history, cancels an open claim (cancelRequst)* |
| Intake Handler/Clerk(Role::Specialist) | Takes ownership of the incoming claim (takeOwnership) and verifies that all required fields are present, and the policy is still active in the registry. |
| Claims Specialist (Role::Specialist) | Executes the category-specific assessment checklist(Stage 2A, Stage 2B or Stage 2C) and records the response_email outcome message in the payload. |
| Life Team Lead (Role::TeamLead) | First level approver(Stage 3A); verifies all the specialist tasks are complete and signs off claims up to R250 000 or escalates claims above that. |
| Senior Claims Manager(Roler::Manager) | High-value approver(Stage 3B); audits payout calculations and regulatory compliance logs to approve(approveRequest) or reject (rejectRequest) claims above R250 000. |

**Input data:**

When a customer submits a claim, Director/ WorkflowEngineFacade populates the WorkItem payload map with 5 required keys.

| Key | Example | How engine uses it |
|---|---|---|
| Policy_number | "POL-99182" | Validated in Stage 1A and SubProcess 1A-1 against the policy registry; persisted in every database audit row. |
| Customer_email | "bob@gmail.com" | Validated in Stage 1A; read by WorkflowEngineFacade to send ResultNotice via EmailChannel. |
| category | "AccidentalClaim" | Read in Step 2: AssessmentHandler to route the claim to Stage 2A, Stage 2B, or Stage 2C. |
| amount | "300000" | Read in Step 3: ApprovalHandler to decide between Stage 3A (<= R250,000) and Stage 3B (> R250,000). |
| description | "Car accident claim – Police Ref #4412" | Checked by Stage 1A and used by specialists during SubProcess 2B-1 / 2C-1 investigation. |

**Decisions and routing points:**

1. Intake validation check (step 1): Are all 5 required fields non-empty and is the policy active> if not, intake rejects the claim and sends a rejection notice
2. Category-Based specialist routing(step 2): Reads payload["category"] to determine which specialist checklist applies. DeathBenefit(Stage 2A), AccidentalClaim(Stage2B and SubProcess 2B-1) and CriticalIllness(Stage2C and subprocesses 2C-1 and 2C-2).
3. Amount based approval routing(step 3): Reads payload["amount"]:
   - <= R250 000: Handled by Stage 3A: TeamLeadApprovalHandler
   - > R250 000: Escalated down the chain (next->handleRequest()) to Stage 3B: SeniorManagerApprovalHandler

**Approvals and Dependencies:**

- Stage-to-stage Dependency (RequestChain::unfinishedGroup): Tasks in Step 2(AssessmentHandler) cant be completed until every task in Step 1 (IntakeHandler) is completed. Same with Step 3 (ApporovalHandler) and Step 2.

- Ownership Dependency (takeOwnership): No specialist can start or complete a task until they have taken ownership of the open claim instance, and no second agent can take an already owned claim.
- Two-tier Financial approval: claims up to R250 000 require Team lead signoff (Stage 3A) and claims above that require Senior Claims Manage reveiw (Stage 3B).

**Completion path:**

1. Claim Submission (submitRequest):
   A customer (Role::Requester) submits the 5 input fields. WorkflowBlueprint::loadAndValidate("src/life_blueprint.json") validates the JSON schema, Director and WorkflowBuilder build and clone (cloneWorkGroups()) an isolated tree of the 3 WorkGroups, stampPayload copies the input map into every component, and attachObserverToTree attaches DatabaseObserver to every node. The claim starts with status "Open".
2. Claim Ownership (takeOwnership):
   A claims specialist (Role::Specialist) takes ownership of the "Open" claim instance (request.owner = actorId), locking it against other specialists.
3. Step 1 - Intake Completion (completeTask on Group 1 tasks):
   The specialist completes the tasks in Step 1: IntakeHandler (Log submission, Verify policy_number & customer_email, Verify category, amount, & description, Verify active policy standing, and Send intake confirmation). For each task, AssignTaskCommand, StartTaskCommand (Pending → In Progress), and CompleteTaskCommand (In Progress → Completed) execute. Each state change calls notify(), and DatabaseObserver writes the transition and payload to Neon Cloud PostgreSQL (or local SQLite). Once all children finish, Step 1 automatically refreshes its own state to Completed, unlocking Step 2.
4. Step 2 - Specialist Assessment Completion (completeTask on Group 2 tasks):
   The claim advances to Step 2: AssessmentHandler, where the specialist completes the category routing task, works through the checklist stages (Stage 2A Death Benefit, Stage 2B Accidental Claim, Stage 2C Critical Illness parallel medical and hospital sub-processes), and completes "Record specialist response_email outcome message in payload map". Step 2 transitions to Completed, unlocking Step 3.
5. Step 3 - Tiered Financial Approval (completeTask on Group 3 tasks + approveRequest):
   In Step 3: ApprovalHandler, Stage 3A: TeamLeadApprovalHandler checks amount against the R250,000 threshold and confirms all specialist tasks are complete. For claims above R250,000 progression continues through Stage 3B: SeniorManagerApprovalHandler (SubProcess 3B-1: Senior Claims Manager High-Value Review). Once all tasks are Completed (chain->isFinished() == true), the approver calls approveRequest(actorId, instanceId), setting the claim status to "Approved".
6. Notification & Audit Trail:
   ResultNotice and DeadlineNotice are dispatched via EmailChannel (using Resend, Brevo, Mailjet, or ConsoleEmailAdapter) and AppChannel, and the full history of states and actions is stored in workflow_audit_log and inspectable with getHistory() and getTaskStates().

**Possible failures and Exception paths:**

- Empty or Invalid Intake (invalid_argument): Submitting an empty payload or an invalid blueprint is immediately rejected at intake.
- Unauthorised Role Action (AccessDeniedError): A customer attempting to complete tasks or approve their own claim, or a team lead attempting to submit a claim, is blocked by AccessControlProxy.
- Out-of-Order Task Execution: Attempting to complete an Assessment or Approval task before Intake finishes is refused by RequestChain::completeTask().
- Claim Rejection (rejectRequest) or Cancellation (cancelRequest): An owned claim that fails contestability/exclusion checks is formally transitioned to "Rejected" with a recorded reason in the history; a customer can also cancel an "Open" claim ("Cancelled"), locking all further task changes.
- Mistaken Task Completion (undoLastAction): If an agent completes a checklist item in error, undoLastAction() uses ActionExecutor and WorkMemento to roll the task state back from Completed to Pending.
- Cloud Database or Email Outage: If Neon Cloud PostgreSQL is unreachable, DatabaseObserver::selectBestStrategy() automatically falls back to local SQLite (tividy_local.db); if email API keys (RESEND / BREVO / MJ) are absent, notification delivery falls back to ConsoleEmailAdapter**.**

**Expected outcome of workflow:**

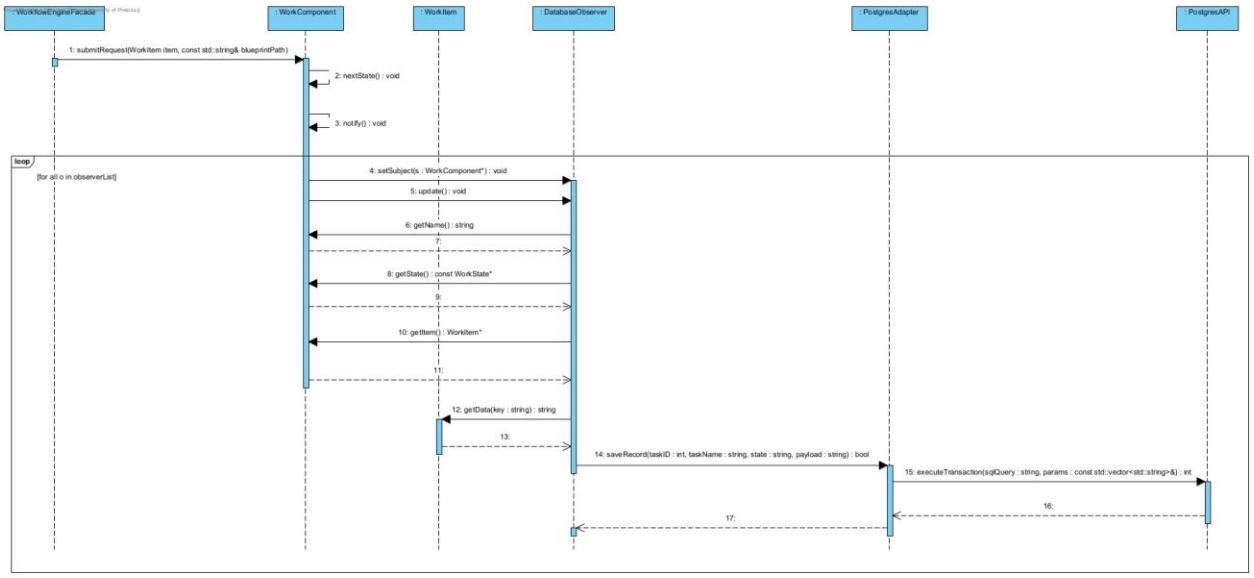
- The claim instance progresses from "Open" to "Approved" (or "Rejected" / "Cancelled" on the exception path),
- Every task and group transition (Pending → In Progress → Completed) is logged with its task_id and full policy_number/category/amount payload into both Neon Cloud PostgreSQL and tividy_local.db,
- The customer and assigned support agent receive ResultNotice and DeadlineNotice notifications via EmailChannel and AppChannel, and the full execution history is inspectable via getHistory() and getTaskStates().

## 7. Runtime behavior

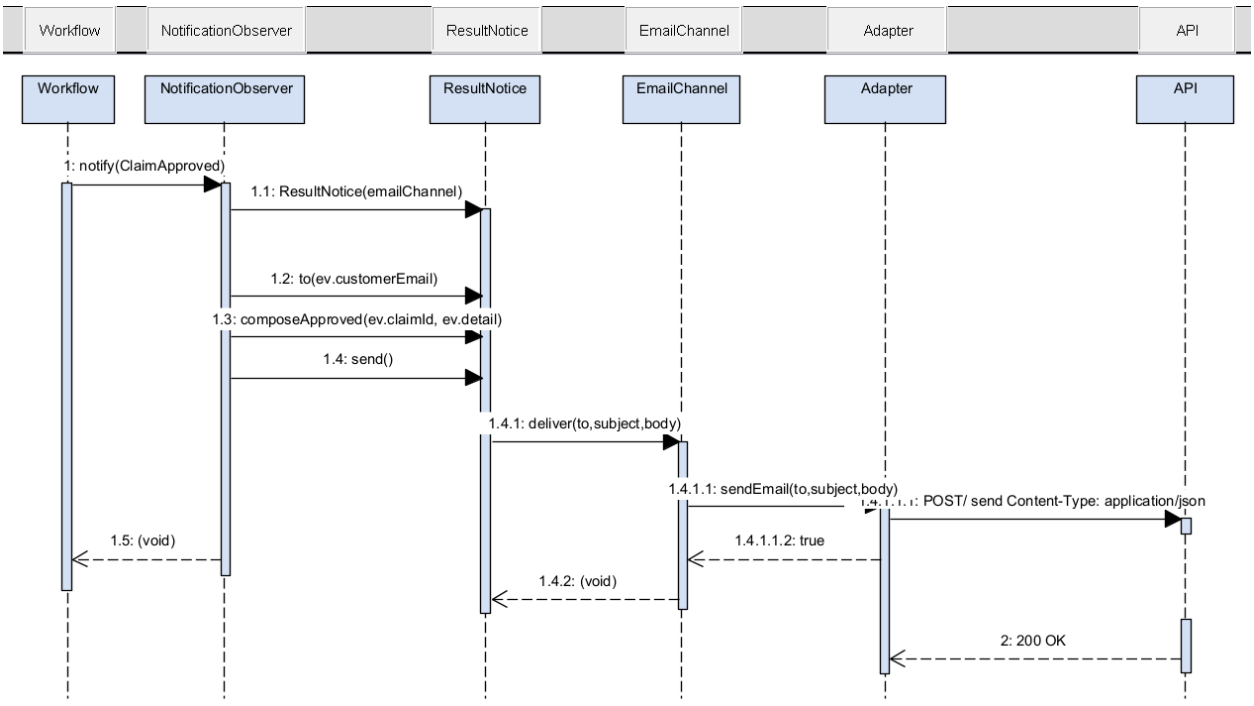
*Insert each diagram with two or three lines on what it shows.*

Sequence 1 and 2 key collaborations:

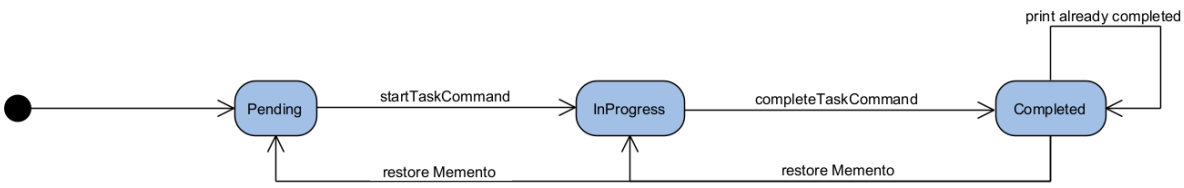
Database Sequence:




Notification Sequence:




State: work item lifecycle



## 8. Build, run and test

### 8.1. Dependencies.

We use Docker to provide a consistent Ubuntu 22.04 environment with all required dependencies, including:

- **CMake**
- **GCC and build tools**
- **GoogleTest**
- **Valgrind**
- **SQLite3**
- **PostgreSQL/libpqxx**
- **CURL**
- **nlohmann-json**

This means the project can run on different laptops without manually installing every dependency.

### 8.2. Build. (docker compose build)

The project is configured and compiled using CMake

```
cmake -B build -DCMAKE_BUILD_TYPE=Debug && \
 cmake --build build -- -j$(nproc)
```

CMake automatically discovers the implementation files in src/ and the test files in tests/.

### 8.3. Run both industries (docker compose run --rm tividy_app)

Select the relevant JSON workflow from the runtime menu. For example:

- University job application workflow
- Student accommodation application workflow

You can also add or swap JSON scenarios in src/ during runtime and refresh the menu without restarting.

**8.4. Run tests.**

Run the complete GoogleTest suite inside Docker:

```
docker compose run --rm tividy_app ./build/bin/tividy_tests
```

Run the memory-leak check:

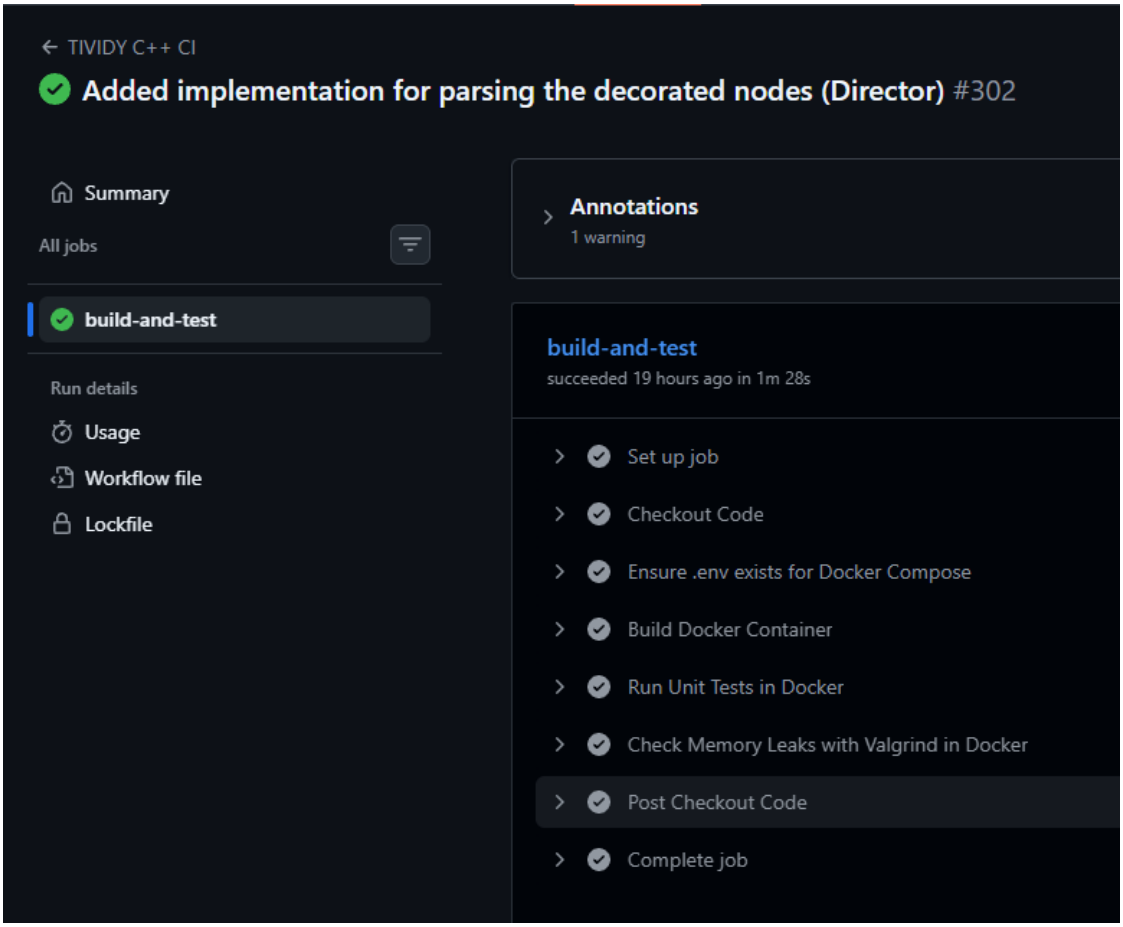
```
docker compose run --rm tividy_app \
  valgrind --leak-check=full \
  --show-leak-kinds=all \
  --error-exitcode=1 \
  ./build/bin/tividy_tests
```

The tests verify the design patterns, workflow construction, runtime scenario validation, Facade behaviour, cloning, input retries, database integration and notifications.

**8.5. Git and CI evidence.**

Our team uses protected main and dev branches. Each feature is developed on a separate branch and merged onto dev through a Pull Request.

- Every push and Pull Request triggers GitHub Actions, which:
- Builds the project inside Docker.
- Runs the unit tests.
- Runs Valgrind to detect memory leaks.
- Blocks merging if any check fails.




## 9. Design decision log

| ID | Question | Decision | Reason |
|---|---|---|---|
| R1 | Why switch from a single GoogleSheetsAdapter (in prac 6) to an adapter and strategy combination for both Postgres and SQLite? | Replaced the single GoogleSheetsAdapter with an Adapter + Strategy combination. ExternalDatabase acts as both the Target (Adapter) and Strategy interface, PostgresAdapter / PostgresAPI wraps Cloud PostgreSQL (libpq), SQLiteAdapter / SQLiteAPI wraps local SQLite (sqlite3), and DatabaseObserver acts as the Strategy Context | A single hard-coded cloud adapter fails if the user is offline or has no cloud credentials. Combining Strategy with Adapter lets DatabaseObserver automatically fall back to local SQLite (selectBestStrategy()) when the cloud is unreachable, or let the user choose either Cloud or Local as their primary database and |

| ID | Question | Decision | Reason |
|---|---|---|---|
| | | holding a primary database and an optional fallbackDatabase. | hot-swap them at runtime (setDatabaseStrategy()). |
| R2 | How do we keep the Cloud PostgreSQL database and local SQLite database consistent after offline work or runtime strategy swaps? | Added fetchAllRecords() to ExternalDatabase and syncDatabases(fromDb, toDb) to DatabaseObserver (and triggered it automatically inside setDatabaseStrategy()). | When the system runs offline on SQLite and later reconnects to Neon Cloud (or swaps from Cloud to Local), syncDatabases() compares existing rows in both databases and copies only the missing records across without creating duplicates. |
| R3 | Why did live Cloud PostgreSQL writes initially take over 2 seconds per row, and how was that solved? | Updated PostgresAPI to keep a single persistent PGconn* handle (dbConn) open across queries until disconnect() called in the destructor ~PostgresAPI(), and added a tableEnsured flag in PostgresAdapter so CREATE TABLE IF NOT EXISTS only runs once per adapter. | Opening a new SSL connection (PQconnectdb) to the neon database in Europe on every saveRecord() call caused multiple seconds onf network handshake delay per task. Reusing the open PGconn* and checking the table had been created once reduced each cloud save dramatically. |
| B1 | Should every workflow be forced into the original three-step sequence of Ingest, Evaluate and Route? | Replaced the fixed three-step Director sequence with a JSON-driven hierarchy of WorkGroup, Stage, SubProcess and WorkUnit nodes. | The engine must support unrelated scenarios such as job applications, accommodation applications and service requests. A fixed sequence would make the supposedly generic engine domain-specific. |
| P1 | Should one static master | Changed to build a | Runtime scenario |

| ID | Question | Decision | Reason |
|---|---|---|---|
| | workflow always be created at startup? | validated blueprint when a request is selected, then clone the constructed workflow for execution. | selection means the active workflow is not known permanently at startup. The same process can select different JSON workflows without restarting. |
| F1 | How should multiple runtime scenarios be selected? | Added WorkflowScenarioCatalog to scan src/*.json, validate available files, list scenarios and refresh the catalog while the application is running. | New scenarios can be introduced during a demo without recompilation or restarting. Invalid files are reported without appearing as selectable options. |
| R6 | What do we do to ensure that the Adapter API remains generic and not too specific? | Added a Strategy pattern for the Adapter to function generically allowing the client to pick their API. | This makes the implementation generic and removes specificity. |

## Appendix A: GoF pattern mapping

| Pattern | Category | Subsystem | Participant classes |
|---|---|---|---|
| Builder | Creational | 5.1 | **Client** (WorkflowEngineFacade), **Director** (Director), **Builder** (WorkflowBuilder), **Product** (Handler), **Concrete Products** (WorkGroup, Stage, SubProcess, WorkUnit) |
| Prototype | Creational | 5.1 | **Client** (WorkflowEngineFacade), **Prototype** (WorkComponent/Handler) , **Concrete Prototypes** (WorkGroup, Stage, SubProcess, WorkUnit) |

| Pattern | Category | Subsystem | Participant classes |
|---|---|---|---|
| Facade | Structural | 5.1 | **Client** (UI), **Facade** (WorkflowEngineFacade), **Subsystems** (Director, WorkflowBuilder, WorkflowScenarioCatalog, WorkflowBlueprint, RequestChain, DatabaseObserver, NotificationBridge) |
| Composite | Structural | 5.2 | Component (WorkComponent), Composite (WorkGroup, Stage, SubProcess), Leaf (WorkUnit) |
| Adapter | Structural | 5.5 | Notification:<br>MailjetAdapter, ResendAdapter, BrevoAdapter,<br>EmailTransport(Adaptee), ControlEmailAdapter, EmailTransport(Abstract Adapter), EmailChannel,<br>SendEmail (Target)<br><br>Database:<br>ExternalDatabase(Target)<br>PostgresAdapter(Adapter)<br>SQLiteAdapter(Adapter)<br>PostgresAPI(Adaptee)<br>SQLiteAPI(Adaptee) |
| Bridge | Structural | 5.5 | Notification (Abstraction), ResultNotice (Refined Abstraction), NotificationChannel |

| Pattern | Category | Subsystem | Participant classes |
|---|---|---|---|
| | | | (Implementor), EmailChannel, InAppChannel(Concrete Implementor) |
| Decorator | Structural | | Component: WorkComponent<br>ConcreteComponent: WorkUnit<br>Decorator: TaskDecorator<br>ConcreteDecorators: ValidatedTask, AuditedTask, PrioritisedTask |
| Proxy | Structural | 5.6 | Subject: WorkflowService<br>RealSubject: WorkflowEngineFacade<br>Proxy: AccessControlProxy |
| State | Behavioural | 5.2 | Context: WorkComponent<br>State: WorkState<br>ConcreteState: PendingState, InCompleteState, CompletedState |
| Iterator | Behavioural | 5.2 | WorkGroup (Aggregate)<br>WorkIterator (Iterator) DepthFirstIterator, FilterIterator, StateIterator, IncompleteIterator, PriorityIterator (Concrete Iterator) |
| Visitor | Behavioural | 5.2 | Element (WorkComponent Concrete Element (WorkUnit, WorkGroup, Stage, SubProcess, PrioritisedTask, AuditedTask, ValidatedTask), Visitor (WorkVisitor), |

| Pattern | Category | Subsystem | Participant classes |
|---|---|---|---|
| | | | ConcreteVisitor (PriorityCheckerVisitor, ProgressReportVisitor, KindFinder) |
| Chain of Responsibility | Behavioural | 5.3 | Hander: Handler<br>ConcreteHandler: WorkGroup, Stage, Subprocess |
| Observer | Behavioural | 5.5 | Notification:<br>WorkflowSubject (Subject), NotificationObserver (Observer)<br>Database:<br>WorkflowSubject(Subject)<br>WorkComponent(ConcreteSubject)<br>WorkflowObserver(Abstract Observer)<br>DatabaseObserver(Concrete Observer) |
| Command | Behavioural | 5.4 | ActionExecutor(Invoker)<br>Action(Command)<br>StartTaskCommand(ConcreteCommand)<br>AssignTaskCommand(ConcreteCommand)<br>CompleteTaskCommand(ConcreteCommand)<br>WorkUnit(Receiver) |
| Memento | Behavioural | 5.4 | WorkUnit(Originator)<br>WorkMemento(Memento) |

| Pattern | Category | Subsystem | Participant classes |
|---|---|---|---|
| | | | WorkHistory(Caretaker) |
| Strategy | Behavioural | 5.5 & 5.7 | Database:<br>DatabaseObserver(Context)<br>ExternalDatabase(Strategy)<br>PostgresAdapter(Concrete Strategy)<br>SQLiteAdapter(Concrete Strategy)<br>Notification:<br>EmailChannel (Context),<br>EmailTransport (Strategy),<br>MailjetAdapter, ResendAdapter, BrevoAdapter, ConsoleAdapter (ConcreteStrategy) |

## Appendix B: References


1. Slack. *What is a workflow?* Link
2. Teiva Systems. *Generic workflow.* Link
3. Workflow Management Coalition. *The Workflow Reference Model*, 1995. Link
4. Camunda. *Escalation Events.* Link
5. ScienceDirect Topics. *Workflow Management System.* Link